

EcoHospital Smart

Ciclo CI/CD com Java Spring Boot

Kalicon Amorim da Cruz Souza — RM 563172

FIAP | Atividade acadêmica | 28/09/2026

Código, testes, containerização e configuração de staging/produção.

Build Java e demonstração HTTP local verificados. Docker, Actions e deploy remoto aguardam execução em infraestrutura real.

Documento gerado a partir de docs/documentacao-tecnica.md. Outros integrantes: preencher se houver.

1. Identificação e objetivo

Integrante: Kalicon Amorim da Cruz Souza — RM 563172. Instituição identificada no projeto original: FIAP.

Outros integrantes: não informados; preencher se houver. Data da preparação: 28/09/2026.

Objetivo: demonstrar build automático, execução de testes existentes, containerização e promoção da mesma versão em staging e produção. A entrega contém código e infraestrutura como configuração; a execução remota permanece pendente de conta, servidor e credenciais.

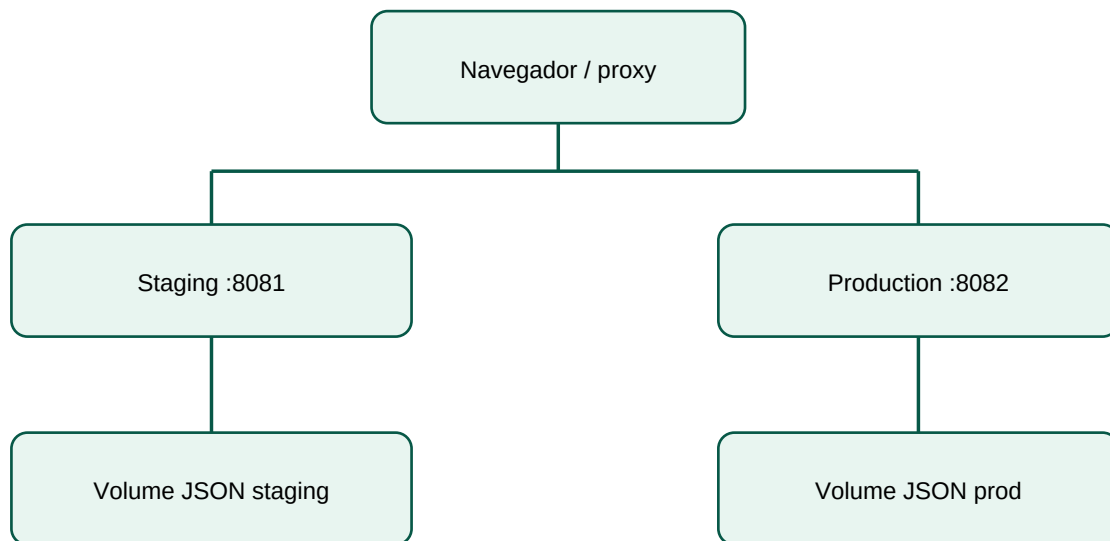
2. Diagnóstico e decisão técnica

O material recebido incluía servidor Node.js nativo, frontend estático, dataset JSON com cinco coleções e scripts acadêmicos MongoDB. Não havia Java/C#, package.json, build, Docker ou CI. A aplicação web simulava consultas MongoDB em arrays e perdia alterações ao reiniciar.

Após autorização para adotar a solução adequada à atividade, foi implementado backend Java 17 com Spring Boot 3.5.16. O frontend e as operações ESG foram mantidos: hospitais, fontes polimórficas, licenças, auditoria, KPIs, consultas, atualizações, simulação IoT e reset. O servidor Node original foi preservado como referência e o runner original continua sendo executado no CI.

Persistência: JSON com gravação atômica em volume por ambiente. Essa escolha mantém o modelo existente com mudanças limitadas e sem serviço de banco artificial. Não se afirma integração MongoDB; seus scripts e prints históricos pertencem à atividade anterior.

3. Arquitetura Docker



Projetos, redes e volumes separados; mesma imagem por digest

Arquitetura prevista: navegador -> proxy/rede do laboratório -> container Spring Boot em cada ambiente -> volume JSON correspondente. O Compose cria uma rede bridge e um volume nomeado por projeto. Staging usa projeto ecohospital-staging e porta 8081; produção usa ecohospital-production e porta 8082. Ambos expõem internamente 8080.

Staging e produção recebem a mesma imagem GHCR por digest SHA256, mas têm configuração e dados separados. Podem estar em dois servidores ou no mesmo host para demonstração, sempre com projetos, portas e volumes distintos. Por padrão, portas no host são limitadas a 127.0.0.1.

O volume contém state.json. O dataset original fica dentro do JAR e é usado somente na criação inicial ou no reset explícito. Reinícios e substituição do container mantêm os dados. Apenas uma instância pode escrever em cada volume. Não há promessa de escalabilidade distribuída, transações MongoDB ou alta disponibilidade.

4. Imagem criada e execução

Dockerfile de dois estágios. Build: maven:3.9-eclipse-temurin-17 executa mvn verify, incluindo JUnit, e gera esg-app.jar. Runtime: eclipse-temurin:17-jre-jammy com JRE, JAR e curl; usuário UID/GID 10001, porta 8080 e healthcheck em /health. Maven e Node não entram no runtime.

O .dockerignore permite apenas pom, Java/testes, frontend e dataset necessário. Compose configura root filesystem somente leitura, tmpfs /tmp, volume de dados gravável, memória de 512 MB, no-new-privileges e capabilities removidas.

Comandos: docker build --build-arg APP_VERSION=academic-local -t ecohospital:local .; docker compose up -d --build --wait --wait-timeout 180. Health esperado: status UP, environment configurado, version da imagem. Esses comandos ainda precisam ser executados em Docker real.

5. Etapas e lógica do pipeline

Gatilhos: pull request, push na main ou execução manual. verify executa checagem JavaScript, runner histórico com asserções e Maven verify. O JAR e os XML JUnit são guardados como artefatos.

image depende de verify aprovado. Constrói a imagem, executa container, confere usuário, health, versão e persistência após reinício. Na main, publica a imagem testada no GHCR usando GITHUB_TOKEN; o digest real é passado aos dois deploys. Pull requests não publicam nem fazem deploy.

staging depende da imagem e de DEPLOY_ENABLED=true. production depende de imagem e staging e exige PRODUCTION_DEPLOY_ENABLED=true. O Environment production deve ter aprovação configurada no GitHub. Falha em teste, container ou health de staging bloqueia produção pelas dependências needs e pelo código de saída das etapas.

Cada deploy usa secrets do seu Environment: SSH_HOST, SSH_USER, SSH_PORT, SSH_PRIVATE_KEY, SSH_KNOWN_HOSTS, GHCR_USER e GHCR_TOKEN. Variables: APP_PORT, BIND_ADDRESS e APP_URL. A verificação SSH do host é estrita; senha/token não são argumentos de linha de comando.

O servidor faz pull, executa Compose com --wait e confere ambiente e SHA no health. O runner consulta a URL externa e confere os mesmos campos. Evidência JSON é arquivada somente a partir da resposta real. Produção usa o digest de staging sem recompilar. Deploys do mesmo ambiente não ocorrem simultaneamente.

6. Evidências reais disponíveis

Build Maven: aprovado com JDK 17.0.16. JUnit: 25 casos, zero falhas, zero erros e zero ignorados. Runner original: asserções de consultas/CRUD aprovadas para 50 documentos. Logs e XML em docs/evidence/local.

HTTP local: dois processos Java com a mesma versão SHA256 do JAR responderam 200 na página e health. Staging recebeu uma leitura IoT e ficou com 11 leituras; produção permaneceu com 10. Após reinício, staging manteve 11. Esses resultados são reais, porém locais e sem containers.

Actionlint 1.7.12 aprovou os dois workflows; Git Bash aprovou sintaxe dos dois scripts de deploy. Isso é análise estática, não execução de GitHub Actions ou SSH. Não houve deploy remoto ou publicação de imagem.

Compose standalone 5.5.1 aprovou config dos ambientes sem daemon. As configurações expandidas confirmam portas 8081/8082, redes e volumes com nomes distintos. Esses recursos ainda não foram criados em Docker. A configuração remota foi validada com imagem/digest de exemplo, sem publicação.

EVIDÊNCIA PENDENTE: print do run GitHub Actions com build/testes, URL real e SHA. Inserir após executar o pipeline.

EVIDÊNCIA PENDENTE: imagem Docker/GHCR, digest e containers/redes/volumes em execução. Inserir após construir/executar Docker.

EVIDÊNCIA PENDENTE: página e health de staging remoto, URL real, ambiente, versão e data. Inserir após deploy real.

EVIDÊNCIA PENDENTE: aprovação de produção, página e health remoto, versão/digest iguais a staging. Inserir após execução e aprovação real.

EVIDÊNCIA PENDENTE: teste propositalmente falho em PR e jobs posteriores bloqueados. Inserir somente o run real.

7. Desafios e soluções

Linguagem incompatível com o pedido: backend portado para Spring Boot, mantendo contrato de endpoints e dados. Ausência de asserções no runner histórico: adicionadas verificações que geram código não zero em divergência. Perda de dados no reinício: snapshots e substituição atômica de arquivo por ambiente.

Primeiro teste da página falhou porque MockMvc retorna forward sem renderizar o destino. Corrigido para verificar forward e index.html; também executado HTTP real. A execução final dos 25 testes passou.

Ferramentas indisponíveis: Maven resolvido com Wrapper e checksum; JDK 17 existente selecionado.

Ausência de Docker e infraestrutura remota: configuração preparada e pendências documentadas.

Nenhuma evidência foi criada para representar execução não realizada.

8. Operação, limitações e entrega

Sem autenticação nas rotas didáticas de mutação/reset; acesso deve ficar limitado ao laboratório ou proxy protegido. Não usar como serviço público de produção sem autenticação e autorização. Google Fonts é externo, com fallback visual do navegador.

Rollback manual preserva volume e usa previous-image.txt do ambiente. Falha de deploy exige diagnóstico; não há rollback automático, zero downtime ou backup remoto automatizado. Dados devem ser copiados antes de mudanças de formato.

Para concluir: criar repo GitHub, Docker Linux, servidor Linux/Compose, conectividade e URLs, secrets, revisão production e habilitação de deploy. Depois executar pipeline, inserir prints reais e marcar apenas itens comprovados. O README contém comandos, tabelas de configuração e checklist.

ZIP inclui código Java/Node, dados, frontend, Docker/Compose, workflows, Wrapper, scripts, exemplos e documentação/evidências disponíveis; exclui segredos, ferramentas e estados runtime. O manifesto SHA256 permite conferir conteúdo.

9. Referências oficiais

Spring Boot 3.5 — requisitos: <https://docs.spring.io/spring-boot/3.5/system-requirements.html>

GitHub — environments e proteção:

<https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments>

GitHub — aprovação e disponibilidade por plano: <https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments/review-deployments>

Anexo — resultados locais reais

Suíte	Testes	Falhas	Erros	Ignorados
EsgApiTest	20	0	0	0
EsgStoreTest	5	0	0	0

HTTP local — staging

```
{"environment": "staging", "version": "981c75e4d3d7aed96803f575c401992f24867c476780e271385a4bcdac120826", "status": "UP"}
```

HTTP local — production

```
{"status": "UP", "version": "981c75e4d3d7aed96803f575c401992f24867c476780e271385a4bcdac120826", "environment": "production"}
```

Verificado em 2026-09-28T10:55:39.2874294-03:00: HTTP 200 em / e /health dos dois processos; versões SHA256 iguais; dados isolados (11/10); persistência após reinício (11). Demonstração local Java, sem Docker ou deploy remoto.

Os resultados anexados são locais. Não representam evidência de Docker ou deploy remoto.